Step 2 — Add a Long-Term Store, and a Node That Reads It — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and how the store reaches the node

Hint 1 — The idea, and the exact pieces

The checkpointer is keyed by `thread_id`, so a new conversation starts empty. The Store is keyed by `user_id`, so it survives across conversations. Add:

- `src/memory/store.py` — `build_store()` (`InMemoryStore`), `namespace_for(user_id)` = `("memories", user_id)`, `recall_facts(store, user_id)`, `save_fact(store, user_id, key, fact)`.
- `recall_node` in `nodes.py` — reads the store, sets `state["recalled"]`.
- `graph.py` — `START -> recall -> chat -> END`.

Save under a **key** (not append): `save_fact(..., "preferred_name", ...)` means a later value replaces the old one — the dedup you'll rely on in Step 3.

Hint 2 — The plan, and how the store reaches the node

```
store.py: put(namespace_for(uid), key, {"fact": fact})
          search(ns) -> items with .key / .value
recall_node(state, *, store):
    return {"recalled": "; ".join(recall_facts(store, state["user_id"]))}
graph.py: START -> recall -> chat -> END; compile(checkpointer, store=store)
```

The one thing to get right: **the node gets the store by declaring a `store` parameter** (`def recall_node(state, *, store):`). LangGraph injects it because you compiled with `store=...`. You don't pass it through the state. Seed a fact by hand, then open a brand-new `thread_id` — `recall_node` still finds it. The whole file set is in the Solution.